An Empirical Study of Proactive Coding Assistants in Real-World Software Development

Lehui Li[*,1,2], Ruixuan Jia[*,1], Guo-Ye Yang[2], Jia Li[†,1]
[1]*College of AI, Tsinghua University*, China
[2]*Fitten Tech Co., Ltd.*, China
[*]Equal contribution [†]Corresponding author

***Abstract*—Large language model (LLM)-based coding assistants have become increasingly capable, yet most remain reactive and provide assistance only after explicit developer instructions. Proactive coding assistants aim to predict developers' implicit intent from integrated development environment (IDE) interaction traces and repository context, thereby reducing the cognitive overhead for writing instructions and improving development efficiency. However, due to the lack of large-scale real-world developer behavior data, existing studies rely heavily on LLM-generated simulated data, whose fidelity to real-world data remains unclear. In this paper, we study this simulation-to-reality gap through large-scale real-world data collection. We collect IDE interaction traces from 1,246 experienced industry developers over three consecutive days, and construct paired LLM-generated simulated traces for controlled comparison. Our analysis shows that LLM-generated simulated traces differ substantially from real-world traces in behavioral diversity, temporal structure, and exploratory behavior. Based on the collected real-world traces, we build ProCodeBench, a benchmark for proactive intent prediction in real-world development scenarios. Experiments on representative LLM, retrieval-augmented, and agent-based baselines show that existing methods remain far from reliable under real-world IDE traces, suggesting that simulation-based evaluation may overestimate real-world intent-prediction performance. Finally, our training study shows that LLM-generated simulated data alone cannot substitute for real-world data, but can improve performance when used before real-world fine-tuning. These findings highlight the importance of real-world developer behavior data for evaluating and training proactive coding assistants, while also revealing the complementary role of LLM-generated simulated data.**

***Index Terms*—proactive code assistance, AI4SE, benchmark, real-world data**

I. Introduction

Recent advances in Large Language Models (LLMs) have substantially improved their performance on software engineering tasks such as code generation [1, 2] and test generation [3], leading to the increasing integration of LLM-based coding assistants into modern software development workflows [4–8]. Their capabilities have expanded rapidly, from early code completion and refactoring [9, 10] to recent coding agents [11] that support multi-turn interaction and autonomous task execution. Despite this progress, most coding assistants still follow a reactive interaction paradigm, providing assistance only after developers issue explicit instructions [12, 13]. This design limits their usefulness in real-world development scenarios. On the one hand, developers must continuously formulate detailed instructions during the coding process, which introduces considerable cognitive overhead [14–16]. More importantly, because software engineering tasks are often complex, developers may struggle to clearly articulate their development intent [14].

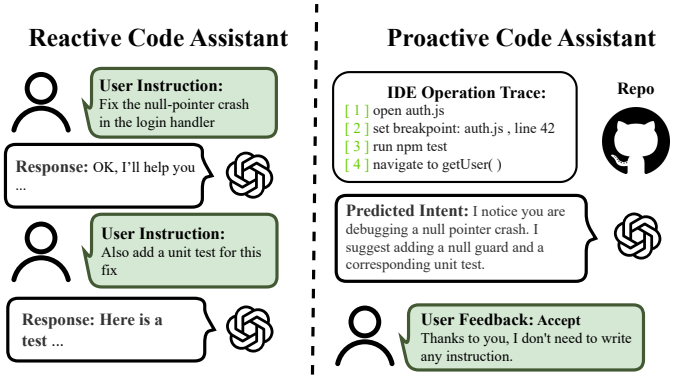

Fig. 1: Comparison of reactive and proactive coding assistants. Reactive coding assistants require explicit instructions for each interaction, while proactive coding assistants predict developers' latent intent from IDE interaction traces, eliminating the need for explicit requests.

To address these limitations, recent studies have proposed proactive coding assistants [12, 13, 17]. As illustrated in Figure 1, unlike reactive coding assistants that wait for explicit instructions, proactive coding assistants infer developers' latent intent from IDE interaction traces and repository context, and then provide corresponding assistance suggestions. Previous user studies have shown that by identifying the intent embedded in developers' IDE interaction traces, proactive coding assistants can improve developers' task performance by 12%–18% on average, while also leading to notable gains in user experience for most participants [14, 15].

However, proactive coding assistants still lack large-scale real-world data on developer behavior [12, 13, 17]. Unlike traditional software engineering tasks, proactive code assistance requires continuous logging of developers' IDE interaction traces in real-world development scenarios [17], which makes data collection costly and subject to strict privacy constraints. As a result, existing studies mostly rely on LLM-generated simulated data [12, 13, 17, 18]. They typically use LLM-based user agents to synthesize IDE interaction traces and generate the corresponding intent labels, which are then used to train and evaluate the intent-prediction ability of proactive coding assistants.

Although LLM-generated simulated data has enabled

progress in proactive coding assistants, it remains unclear whether such data reflects how developers behave in real-world development scenarios. This motivates our first research question:

> ✎ **RQ1:** *Can LLM-generated simulated data faithfully capture real-world developer behavior and the underlying intent?*

To answer this question, we collected over 4 million real-world IDE interaction traces through a Visual Studio Code (VS Code) extension. The data covers 1,246 volunteers over three consecutive days and spans representative development scenarios, including frontend, backend, database, and algorithm engineering. For each real-world trace, we further synthesize a paired LLM-generated simulated trace, which enables a controlled comparison between real-world and LLM-generated simulated data. Our gap analysis reveals a clear simulation-to-reality gap. Real-world traces show greater behavioral diversity, finer-grained operations, and more complex temporal patterns. They also contain richer but noisier process-level information. In contrast, LLM-generated simulated traces are easier to generate, but they often follow simplified behavioral patterns and fail to approximate real development processes.

Because simulated and real-world data differ substantially, evaluations based on simulated benchmarks may overestimate models' ability to proactively predict user intent. We therefore further examine:

> ✎ **RQ2:** *How do existing proactive coding assistants actually perform when evaluated on real-world data?*

To answer RQ2, we construct ProCodeBench from the collected real-world IDE interaction traces. Because developer intent is implicit in continuous IDE interaction traces, we convert raw traces into standardized intent-prediction instances through an annotation pipeline. We then conduct a broad evaluation of 13 competitive baselines, covering seven current LLMs (e.g., GPT-5.4, Claude Sonnet 4.6, and Gemini 3.1 Pro), four Retrieval-Augmented LLMs (e.g., RepoCoder and RepoGraph) [19, 20], and two LLM-based Agents (SWE-Agent and A-RAG) [11, 21]. The results reveal three findings. ❶ Current baselines still struggle to predict developer intent from real-world IDE interaction traces, with performance substantially below that reported on simulation-based benchmarks [17, 18]. ❷ Repository-level code context consistently improves intent-prediction performance across backbone models, indicating that repository information helps clarify the purpose behind observed IDE operations. ❸ LLM-based Agents achieve the strongest results through multi-turn tool use, but how to use repository-level code context effectively and efficiently remains an open challenge.

The poor real-world performance observed in RQ2 raises a further question:

> ✎ **RQ3:** *Can training on simulated or real-world data improve proactive intent prediction?*

To answer RQ3, we compare models trained with real-world data, LLM-generated simulated data, and a mixed-data training regime under a unified setting. The results show that training on LLM-generated simulated data alone does not transfer well to real-world development scenarios. However, the mixed-data training regime improves real-world performance when LLM-generated simulated data is used as an initialization before fine-tuning on real-world data. This suggests that real-world and LLM-generated simulated data are not interchangeable substitutes, but complementary data sources for improving proactive intent prediction. Our main contributions are as follows:

- ❶ We present the first large-scale real-world dataset for proactive code assistance. The dataset contains three consecutive days of IDE interaction traces from 1,246 volunteers across representative development scenarios. By pairing each real-world trace with an LLM-generated simulated counterpart, we reveal a clear simulation-to-reality gap in developer behavior, including differences in behavioral diversity, operation granularity, temporal patterns, and process-level noise.
- ❷ We build **ProCodeBench**, the first benchmark that evaluates proactive code assistance in real-world development scenarios. We convert raw IDE interaction traces into standardized intent-prediction instances through an annotation pipeline, and provide a unified evaluation protocol. Experiments on mainstream LLMs, Retrieval-Augmented LLMs, and LLM-based Agents show that existing models still struggle with real-world proactive intent prediction.
- ❸ We analyze how real-world data, LLM-generated simulated data, and a mixed-data training regime contribute to model training. Our results show that LLM-generated simulated data alone does not generalize well to real-world development scenarios, but it can improve performance when used as an initialization before fine-tuning on real-world data. This finding suggests that LLM-generated simulated and real-world data serve complementary roles rather than interchangeable ones.

II. Related work

LLM-based coding assistants. LLM-based coding assistants have advanced rapidly across software engineering tasks, from code completion and generation [9, 10, 22] to program repair [23], automated test generation [3], and repository-level code comprehension [24]. SWE-Agent [11] further introduced multi-turn reasoning with tool-use capabilities, enabling models to autonomously navigate codebases, retrieve context, and execute repairs. Meanwhile, commercial coding assistants such as Cursor, GitHub Copilot, and Windsurf have become deeply integrated into the IDE, forming an integral part of users' development workflows. Despite this progress, all existing systems remain fundamentally reactive—providing assistance only upon receiving an explicit user query and unable to intervene when the user's intent has not yet been articulated. Proactive code assistance, the focus of this work, aims to overcome this limitation [12, 13, 17].

Proactive assistance. Recent research has begun to explore proactive assistance [12, 13, 17, 18, 25], where the system predicts a user's latent intent from behavioral sequences and context and proactively offers suggestions without an explicit query. ProActiveAgent [17] first formalized the proactive assistance task and constructed ProActiveBench via LLM-based simulation, covering coding, writing, and other scenarios. ProperSim [18] further extended the simulation framework to daily-life scenarios. CodingGenie [13] prototyped a proactive coding assistant within VS Code and evaluated its interaction design via user studies, without systematic benchmarking. A common limitation of the above work is that their training and evaluation data rely entirely on LLM-generated simulated IDE interaction traces [17, 18, 26]. Notably, several concurrent efforts [27, 28] have improved data realism by incorporating real screenshots captured from users' devices. However, screenshots only capture instantaneous interface states and lack fine-grained operational signals—edit deltas, terminal output, cursor movements—as well as repository-level code context. Moreover, these approaches primarily target general Graphical User Interface (GUI) scenarios, making them ill-suited for coding-specific proactive assistance. In contrast, this work presents the first proactive code assistance benchmark derived from real-world IDE interaction traces.

Benchmarks and datasets for real-world software development. A wide range of benchmarks have been established for software engineering research [29, 30]. HumanEval [1] and MBPP [31] evaluate function-level code generation, SWE-bench [32] extends evaluation to the repository level, and CodeSearchNet [33] and DevBench [34] provide large-scale data for code comprehension and maintenance. All of these benchmarks, however, take static code or natural-language descriptions as input [1, 31–34]. Proactive code assistance, by contrast, operates on dynamic IDE interaction traces produced by users during development—a data modality absent from existing software engineering benchmarks. ProCodeBench fills this gap as the first benchmark to incorporate real-world IDE interaction traces into the training and evaluation of proactive code assistance.

III. Task Definition

Unlike reactive coding assistants that rely on explicit instructions, proactive coding assistants aim to predict a developer's implicit intent from the developer's IDE interaction trace and repository context, and to provide assistance before the developer issues an explicit instruction. Formally, we define an IDE operation as a user action recorded during development, such as editing code, switching views, selecting code, or executing a terminal command. Each operation is represented as a tuple $o_i = (p_i, g_i, c_i, t_i)$, where p_i denotes the operation type; g_i denotes the target entity, such as a file, function, or selected code region; c_i denotes the operation content; and t_i denotes the timestamp. A sequence of n consecutive IDE operations forms an IDE interaction trace $\mathcal{O} = \{o_1, o_2, \ldots, o_n\}$. Given this trace $\mathcal{O}$ and the repository-level code context $\mathcal{C}$, the task is to predict the developer's intent:

$$\text{Intent} = f_\theta(\mathcal{O}, \mathcal{C}),$$

where f_θ is the proactive coding assistant parameterized by θ. The output is a natural-language intent description. Figure 2 presents a representative example: after observing that a developer defines a `retry_with_backoff` decorator and then inspects several API-calling modules, the assistant is expected to predict the intent of applying the decorator across the dataflow layer.

A representative example from ProCodeBench

Input — IDE interaction trace $\mathcal{O}$ *(developer working on a Python project)*

o_1 COPY "*handle API call error and retry; review* `cli/*.py` *and* `tradingagents/*.py`"
o_2 VIEW open `dataflows/utils.py`
o_3 EDIT in `utils.py`: add `import time, functools`; define `retry_with_backoff(retries, backoff)` decorator
o_4 VIEW open `dataflows/alpha_vantage_common.py` (contains API functions)
o_5 EDIT in `alpha_vantage_common.py`: add `import time`
o_6 VIEW switch back to `utils.py` to inspect the decorator
o_7 VIEW switch back to `alpha_vantage_common.py`

Output — predicted intent $\text{Intent} = f_\theta(\mathcal{O}, \mathcal{C})$

Apply the newly defined `retry_with_backoff` *decorator to API-calling functions across the dataflow modules.*

Fig. 2: A representative example from ProCodeBench. The IDE interaction trace—copying a note about API retries, defining a `retry_with_backoff` decorator in `utils.py`, then opening API-calling modules—naturally suggests the latent intent of *applying the new decorator across the dataflow layer*. Operation types: COPY, VIEW, EDIT.

IV. Research Methodology

As shown in Figure 3, our research methodology comprises three stages, each addressing one research question. **Stage 1: data collection and gap analysis** collects real-world IDE interaction traces from senior engineer volunteers and pairs each with an LLM-generated simulated counterpart, then quantifies the distributional gap between the two data sources. **Stage 2: benchmark construction** reconstructs the collected real-world IDE interaction traces into a standardized benchmark for proactive code assistance through an intent annotation pipeline, equipped with a unified evaluation protocol and representative baselines. **Stage 3: training analysis** investigates the respective roles of real-world and LLM-generated simulated data in model training through a controlled comparison of different training regimes on the benchmark.

A. Stage 1: data collection and gap analysis

To answer RQ1 (*can LLM-generated simulated data faithfully capture real-world developer behavior and the underlying*

TABLE I: Eight IDE operation types captured by our VS Code extension. Each operation record is stored as a timestamped JSON record with the listed fields.

Operation type	Captured information
`edit`	File path, the inserted and deleted text segments, and the surrounding code context with line range
`copy/paste`	Copied or pasted text content from the system clipboard
`view switching`	File path, the viewport line range, and the actual code content visible within the viewport
`cursor_selection`	File path, the cursor position or selected text span, and the surrounding code context with a cursor marker
`terminal_execution`	Shell command line, exit code, execution duration, and the captured terminal output
`debug`	Debug session identifier, output category (e.g., `stdout`/`stderr`), and the captured debug output
`code_completion`	File path, line number, the accepted completion text, and the surrounding code context
`agent_request`	The natural-language request issued by the developer to the coding agent

intent?), we need to obtain large-scale, realistic IDE interaction traces from volunteers, together with paired LLM-generated simulated traces that enable a controlled comparison.

Real-world data collection. To collect real-world developer behavior data, we first develop a VS Code extension that records a broad spectrum of IDE operations. As summarized in Table I, the extension captures eight operation types that cover the most common user-IDE operations, including editing, copy/paste, view switching, cursor selection, terminal execution, and debugging. In particular, it also records AI-assisted development operations, including accepted code completions through `code completion` and natural-language requests sent to coding agents through `agent request`. Each operation record is stored as a timestamped JSON record with a structured payload. For example, an `edit` operation records the edited file path, the inserted and deleted text spans, and the surrounding code context before and after the edit.

Furthermore, we recruit 1,246 experienced industry developers as volunteers, covering five major development scenarios: backend development (412, 33.1%), frontend development (287, 23.0%), full-stack development (208, 16.7%), algorithm engineering (183, 14.7%), and database development (156, 12.5%), as summarized in Table II. Over three consecutive days, each volunteer works on their own actively maintained industrial project with our VS Code extension enabled. All volunteers receive monetary compensation after completing the collection. In total, we collect approximately **4.63 million operation events** from real-world IDE interaction traces.

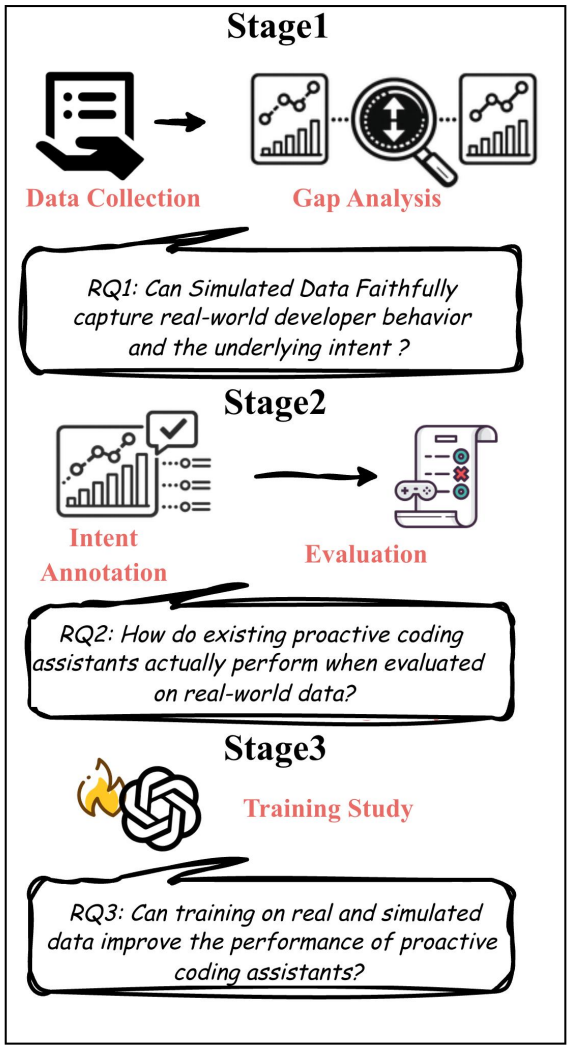

Fig. 3: Overview of our research methodology. Stage 1 collects paired real-world and LLM-generated simulated IDE interaction traces for gap analysis. Stage 2 constructs ProCodeBench by annotating real-world traces as intent-prediction instances. Stage 3 compares training regimes based on real-world data, LLM-generated simulated data, and a mixed-data training regime.

LLM-generated simulated data construction. To construct paired synthetic data, we follow the pipeline of ProActiveAgent [17]. For each real-world IDE interaction trace, we generate one corresponding LLM-generated simulated trace. To ensure a fair comparison, the simulator is given the same volunteer profile information, including job background, development experience, and primary technology stack. We also constrain each LLM-generated simulated trace to match its paired real-world trace in length and to use the same eight operation types.

Gap analysis. With the paired real-world and LLM-generated simulated IDE interaction traces, we analyze their distributional

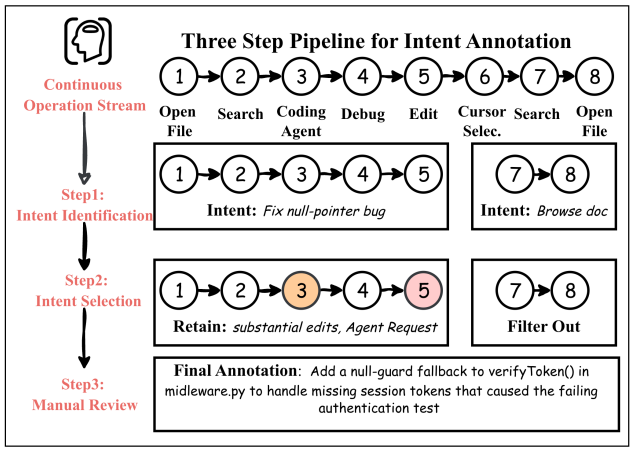

Fig. 4: Three-step intent annotation pipeline for converting continuous real-world IDE interaction traces into standardized intent-labeled evaluation samples.

gap from three perspectives: *behavioral diversity*, *temporal patterns*, and *noise patterns*. *Behavioral diversity* is measured by the frequency distribution of operation types, indicating whether LLM-generated simulated traces cover the long-tail IDE operations observed in real development processes. *Temporal patterns* are characterized through inter-operation time intervals and transition matrices over operation types, revealing whether simulation reproduces the multi-scale pattern of real coding processes and the frequent switches across operation types. *Noise patterns* refer to operations only weakly related to the final intent, such as speculative file browsing, redundant navigation. We further use a representative case study to illustrate how such noise patterns manifest differently in real-world and LLM-generated simulated traces.

B. Stage 2: benchmark construction

To answer RQ2 (*how do existing proactive coding assistants perform on real-world data?*), we construct a standardized evaluation benchmark from the collected real-world IDE interaction traces. Unlike LLM-generated simulated data, real-world traces do not come with explicit intent labels. Moreover, developer intent is only implicitly reflected in continuous IDE operations, where a developer may move across different intents, such as refactoring a function or debugging an exception, without clear boundaries between them. Therefore, the main challenge is to convert continuous IDE interaction traces into evaluation instances. To this end, we design a three-step annotation pipeline consisting of intent identification, intent filtering, and manual review (Figure 4). Based on the resulting intent-labeled instances, we build a unified evaluation protocol and a set of representative baselines.

Step 1: Intent identification. To identify developer intents from continuous IDE interaction traces, we adopt a sliding-window strategy. Each window contains N consecutive operations, from which an LLM identifies the latent intents, locates their starting and ending operations, and assigns an initial natural-language intent annotation to each. We set $N = 50$ to balance identification quality against LLM annotation cost.

Step 2: Intent filtering. To select evaluation-worthy intent segments, we further filter the candidates identified in Step 1. Since developers do not explicitly specify which intents would be particularly valuable for assistance, we use observable behavioral signals as proxies. We adopt a two-step filtering strategy. The heuristic filtering retains candidate intents with substantial code edits or explicit AI-assistant requests, which often indicate complex development tasks that may benefit from assistance. The semantic filtering uses an LLM to examine whether each retained segment is coherent and consistent with its intent description.

Step 3: Manual review. To ensure annotation quality, we further conduct a manual review after the automated identification and filtering steps. These automated steps may introduce two types of errors: some retained candidates may have inaccurate intent descriptions, while some valid candidates may have been incorrectly filtered out. Two domain experts independently inspect the candidates, correct erroneous intent descriptions, and recover valid candidates from the filtered set.

Dataset split. After the three-step annotation pipeline, we obtain 5,492 valid evaluation samples. To avoid temporal data leakage, we split the samples chronologically into training, validation, and test sets, containing 3,576, 1,142, and 774 samples, respectively (see Table II). Each evaluation sample takes the preceding IDE interaction trace as input and the corresponding natural-language intent description as output.

Evaluation protocol. To evaluate the intent-prediction ability of proactive coding assistants, we compare model-generated intents with the developer's real intent. Since developer intents are represented as natural-language descriptions, exact matching is insufficient for evaluating semantic correctness. We therefore adopt an **LLM-as-a-Judge** evaluation strategy [35]. For each sample, an independent LLM judge receives the model-generated intent description and the developer's real intent, and determines whether the two are semantically equivalent.

TABLE II: ProCodeBench dataset statistics. 1,246 volunteers were tracked over 3 consecutive days, yielding 5,492 annotated samples.

Data split	Samples	Ratio
Train	3,576	65.1%
Validation	1,142	20.8%
Test	774	14.1%

Developer domains	Volunteers	Ratio
Backend	412	33.1%
Frontend	287	23.0%
Full-stack	208	16.7%
Algorithm	183	14.7%
Database	156	12.5%

Unlike prior work that relies entirely on LLM-based judgment, ProCodeBench provides developers' real intents as ground truth, which helps mitigate potential bias from the LLM judge [36]. We use **Pass@**K as the primary metric: for each sample, the model independently generates K intent descriptions, and the prediction is considered correct if at least one of them is judged semantically equivalent to the developer's real intent.

Baselines. We compare the following baseline methods on ProCodeBench.

LLMs. To evaluate the intent-prediction ability of current frontier LLMs, we select several widely used models from recent general and software-engineering benchmarks, including DeepSeek-V3.2, GLM-5 [37], MiniMax-M2.5, Qwen3.5-397B [38], GPT-5.4, Claude Sonnet 4.6, and Gemini 3.1 Pro. These models take the developer's IDE interaction trace as input and directly predict the developer's intent.

Retrieval-Augmented LLMs. To evaluate how models use repository-level code context, we include Retrieval-Augmented Generation (RAG)-based methods. Non-graph methods, including RepoCoder [19] and CodeRAG [39], retrieve relevant code fragments through text-based or embedding-based retrieval. Graph-based methods, including GraphCoder [40] and RepoGraph [20], further incorporate repository structure through static code relations, such as call dependencies, import relations, and symbol-level links.

LLM-based Agents. We also evaluate LLM-based Agents, including SWE-Agent [11] and A-RAG [21]. Rather than relying on a single retrieval step, these agents obtain repository context through tool-based interaction, such as browsing files, searching code, and inspecting symbols over multiple turns before producing the intent prediction. It is worth noting that previous proactive coding assistants can be viewed as extensions of LLM-based Agents, with an additional capability for predicting user intent proactively. Therefore, we do not treat prior proactive coding assistants as a separate baseline category.

C. Stage 3: training analysis

To answer RQ3 (*can training on simulated or real-world data improve proactive intent prediction?*), we compare the performance of models trained on each data source individually and on the two sources jointly.

Training regimes. We compare three training regimes to study how real-world and LLM-generated simulated data contribute to model training. **+Real** fine-tunes the backbone on the real-world training set, and **+Sim.** fine-tunes it on the paired LLM-generated simulated training set. **+Sim.→Real** denotes a mixed-data training regime, where the model is first trained on LLM-generated simulated data and then further fine-tuned on real-world data. All regimes are evaluated on the same real-world validation and test sets.

Backbone models. We conduct fine-tuning experiments with open-source models that can be trained under our computational budget of 8×A800-80GB GPUs. To compare training effects across different model families at a similar parameter scale, we select three LLMs: **Qwen-3-8B** [41], **GLM-4-9B** [42], and **LLaMA-3-8B** [43].

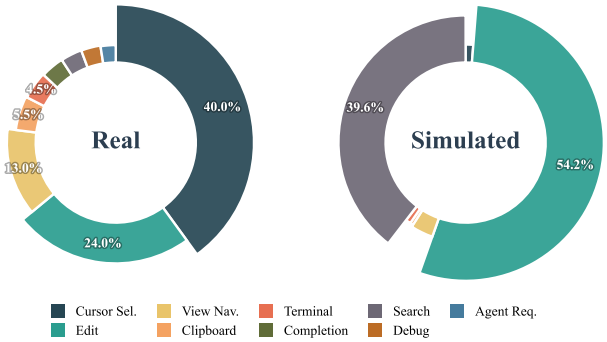

Fig. 5: Operation type frequency distribution. Real-world data covers a broader range of operation types with a pronounced long-tail pattern, while LLM-generated simulated data concentrates on a few high-frequency categories.

V. Results

In this section, we report experimental results organized around the three research questions.

A. RQ1: Distributional gap between real-world and LLM-generated simulated IDE interaction traces

We aim to reveal the distributional gap between real and simulated development data, which constitutes one of the core motivations of ProCodeBench. We analyze this gap from three complementary perspectives: behavioral diversity, temporal patterns, and noise patterns.

Behavioral diversity. As shown in Figure 5, real-world and LLM-generated simulated traces differ substantially in their operation-type distributions. More specifically, LLM-generated simulated traces are concentrated on a small set of operations, mainly code editing and navigation, whereas real-world traces cover a broader range of IDE operations. Consequently, several operation types that regularly appear in real development processes, including AI-assisted operations, are much less frequent in simulation. Notably, the real-world distribution shows a distinctive pattern: **cursor selection (∼40%)** and **view switching (∼13%)** account for the largest proportions, rather than code editing. This suggests that developers spend considerable time reading, inspecting, and navigating code before making modifications. Taken together, these observations indicate that LLM-based simulators tend to generate a limited set of operation types, while overlooking the diversity of operations involved in real-world development scenarios.

Temporal patterns. To reveal how real-world and LLM-generated simulated traces differ in temporal patterns, we analyze both inter-operation intervals and operation-type transitions. For inter-operation intervals, real-world traces exhibit a pronounced bimodal distribution, with one peak near 0.1 s and another around tens of seconds (Figure 6). This pattern reflects the multi-scale temporal structure of real

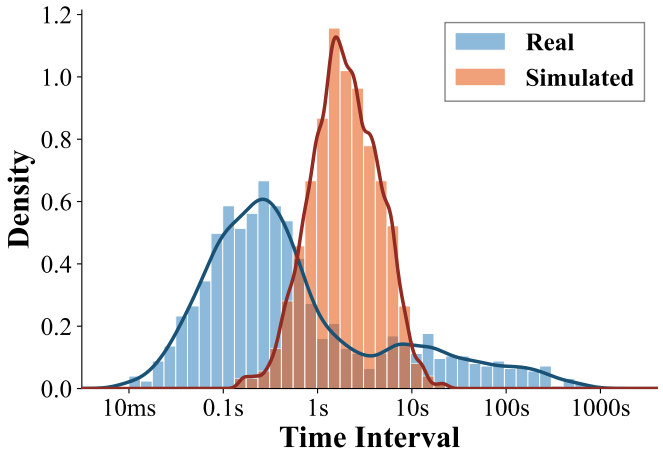

Fig. 6: Distribution of inter-operation time intervals. Real-world data exhibits a bimodal pattern with peaks near 0.1s and tens of seconds, while LLM-generated simulated data shows a unimodal distribution centered around 1s.

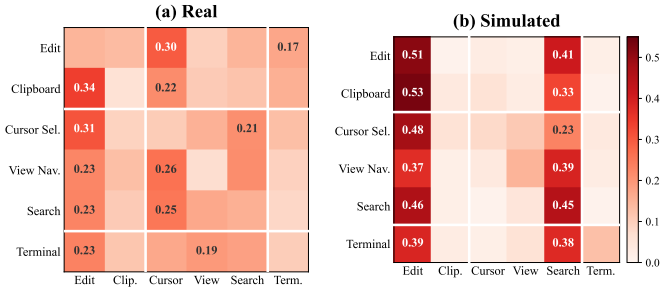

Fig. 7: Operation-type transition patterns in real-world and LLM-generated simulated data. Real-world traces involve diverse transitions across operation types, whereas LLM-generated simulated traces concentrate on a small set of frequent transitions.

development, where short bursts of operations are interleaved with longer pauses. By contrast, LLM-generated simulated traces show a unimodal distribution centered around 1 s, indicating a more regular temporal structure. For operation-type transitions, real-world traces spread across many pairs of operation types (Figure 7), suggesting frequent switches among different development activities. However, LLM-generated simulated traces concentrate on a small number of adjacent transition pairs, such as Edit→Edit and Edit→Navigation. Taken together, these results indicate that LLM-based simulators do not adequately reproduce the temporal structure and operation-switching patterns of real coding processes.

Noise patterns. Real-world traces contain many operations that are only weakly related to the final intent, since developers often adjust their actions while exploring and refining their solution. As illustrated in Figure 8, these operations may include irrelevant browsing, repeated navigation, or reverted edits. Although they introduce noise patterns, they also reflect the uncertainty and nonlinearity of real development processes.

Case study: noise patterns in real-world vs. LLM-generated simulated IDE interaction traces

Real-world IDE interaction trace (intent: *fix null-pointer crash in login handler*)

[1] open `auth.js`	✓
[2] navigate to `handleLogin()`	✓
[3] set breakpoint `auth.js:42`	✓
[4] run `npm run dev`	✓
[5] open `config.js`	✗ redundant
[6] open `README.md`	✗ redundant
[7] back to `auth.js`	✗ redundant
[8] navigate to `userService.js` → `getUser()`	✓
[9] back to `auth.js`	✗ redundant
[10] edit `handleLogin()`: add null guard	✓
[11] undo last edit	✗ redundant
[12] edit `handleLogin()`: add guard with fallback	✓
[13] run `npm test`	✓
[14] select test output: `1 failed`	✓
[15] edit `handleLogin()`: fix edge case	✓
[16] run `npm test`	✓

LLM-generated simulated IDE interaction trace (intent: *add retry logic to API request*)

[1] open `api.js`	✓
[2] navigate to `fetchData()`	✓
[3] edit `fetchData()`: add retry wrapper	✓
[4] edit `fetchData()`: add max retry config	✓
[5] open `api.test.js`	✓
[6] edit `api.test.js`: add retry test case	✓
[7] run `npm test`	✓

Fig. 8: Representative noise patterns. The real-world IDE interaction trace contains exploratory browsing (`[3--4]`), redundant navigation (`[5]`), and a reverted edit (`[7]`), whereas the LLM-generated simulated IDE interaction trace proceeds linearly without trial-and-error.

By contrast, LLM-generated simulated traces rarely contain similar exploratory or corrective behavior. Instead, they tend to generate simplified action sequences. Taken together, this contrast indicates that real-world traces contain human exploratory noise, whereas LLM-generated simulated traces exhibit an oversimplification bias.

Key finding ❶: *LLM-generated simulated traces fail to faithfully capture real-world developer behavior. Compared with real-world IDE interaction traces, LLM-generated simulated traces show reduced behavioral diversity, more regular temporal patterns, and omit much of the exploratory behavior present in real coding processes.*

B. RQ2: Performance of existing proactive coding assistants on real-world data

LLM results. As shown in Table III, all LLMs achieve limited performance on the real-world test set, with Pass@1 below 14% across all models. Even the strongest model, Claude Sonnet 4.6, reaches only 13.57%. This result indicates that current frontier LLMs have limited ability to predict developer intent from real-

TABLE III: LLM Pass@K accuracy (%) on ProCodeBench (759 samples). **Red**: best, Blue: runner-up.

Model	Pass@1	Pass@3	Pass@5
Claude-Sonnet-4-6	**13.57**	**21.61**	**24.37**
Gemini-3.1-Pro	11.46	17.79	21.87
Qwen3.5-397B	8.43	15.68	16.21
GLM-5	7.77	12.65	15.42
DeepSeek-V3.2	7.77	13.44	19.24
GPT-5.4	6.59	16.34	19.10
MiniMax-M2.5	2.77	6.46	7.91

world IDE interaction traces. Compared with the substantially stronger results reported on simulation-based benchmarks [17, 18], the results further suggest that simulation-based evaluation may overestimate real-world intent-prediction performance.

We also observe that model rankings on ProCodeBench do not align with those commonly observed on general software-engineering benchmarks [32]. For instance, GPT-5.4 achieves 6.59% Pass@1, lagging behind Claude Sonnet 4.6 (13.57%) and Gemini 3.1 Pro (11.46%). This suggests that proactive intent prediction requires capabilities beyond general reasoning, and improving this ability remains an open challenge.

Retrieval-Augmented LLM and LLM-based Agent results. Across all backbone models, Retrieval-Augmented LLMs and LLM-based Agents outperform the corresponding LLMs, indicating that IDE interaction traces alone are often insufficient for proactive intent prediction. For example, with GLM-5 as the backbone, Pass@1 increases from 7.77% to 9.49%–16.73% after repository context is introduced. Similar gains are observed for GPT-5.4, DeepSeek-V3.2, and Qwen3.5.

The improvement is especially pronounced for LLM-based Agents. Compared with Retrieval-Augmented LLMs, LLM-based Agents achieve the highest Pass@1 across all four backbones. For instance, A-RAG reaches 16.73% with GLM-5, 35.57% with GPT-5.4, 15.81% with DeepSeek-V3.2, and 15.02% with Qwen3.5. This indicates that autonomous multi-turn tool use is important for proactive intent prediction.

However, this performance gain comes with a substantial efficiency cost. LLM-based Agents require an average of 23 tool interactions per prediction, which increases computation and response latency. Developing methods that achieve both stronger performance and improved efficiency remains an important direction for future proactive code assistance.

Key finding ❷: *Existing models perform poorly on real-world benchmark, suggesting that simulation-based benchmarks may overestimate their intent-prediction ability. Repository-level code context consistently improves performance, but obtaining useful context efficiently remains a key challenge.*

C. RQ3: Training study

To answer RQ3, we compare three training regimes on three open-source backbones: Qwen-3-8B, GLM-4-9B, and LLaMA3-8B. The three regimes include fine-tuning on LLM-generated simulated data only (+Sim.), fine-tuning on real-world data only (+Real), and a mixed-data training regime (+Sim.→Real). All regimes are evaluated on the same real-world test set with Pass@1. We also track the training loss to examine the optimization behavior of different regimes. Results are reported in Table V and Figure 9.

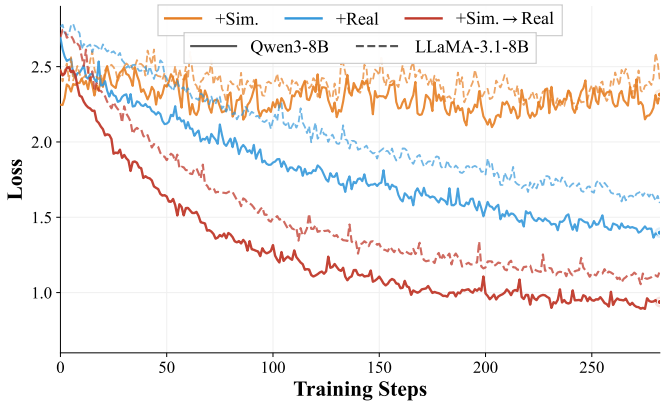

Fig. 9: Training loss curves during fine-tuning on real-world data under different training regimes. +Sim.→Real converges significantly faster and reaches a lower final loss than +Real.

Single-source training. As shown in Table V, real-world data provides a clear benefit for model training, whereas LLM-generated simulated data alone does not transfer effectively to the real-world test set. Fine-tuning on real-world data (+Real) consistently improves all three backbones, increasing Pass@1 from 2.53% to 5.84% for Qwen-3-8B, from 2.24% to 4.93% for GLM-4-9B, and from 1.84% to 3.76% for LLaMA3-8B. In contrast, fine-tuning on LLM-generated simulated data alone (+Sim.) reduces performance below the original backbone, with Pass@1 dropping to 1.97%, 1.65%, and 1.32%, respectively. These results suggest that the gap between LLM-generated simulated and real-world traces directly affects training: models trained only on LLM-generated simulated data fail to generalize to real-world proactive intent prediction.

Complementarity between simulated and real-world data. Although LLM-generated simulated data performs poorly as a standalone training source, it becomes useful when followed by real-world fine-tuning. The mixed-data training regime (+Sim.→Real) achieves the best Pass@1 across all three backbones, reaching 7.63% on Qwen-3-8B, 6.52% on GLM-4-9B, and 5.21% on LLaMA3-8B. Compared with +Real, this corresponds to gains of 1.79, 1.59, and 1.45 percentage points, respectively. The training loss curves in Figure 9 further show that models initialized with LLM-generated simulated data converge faster and reach a lower final loss during real-world fine-tuning. These results suggest that LLM-generated simulated data can provide a useful warm start, while real-world data is still necessary to adapt the model to real-world development scenarios. Therefore, LLM-generated simulated and real-world data should not be viewed as interchangeable sources, but as complementary signals for training proactive coding assistants.

TABLE IV: Pass@1 accuracy (%) of Retrieval-Augmented LLMs and LLM-based Agents on ProCodeBench (759 samples). **Red**: best, Blue: runner-up per row.

Model	Retrieval-Augmented LLMs				LLM-based Agents	
	RepoCoder	CodeRAG	GraphCoder	RepoGraph	SWE-Agent	A-RAG
GLM-5	9.49	10.01	11.07	10.54	14.23	**16.73**
GPT-5.4	10.67	11.46	14.36	13.57	32.98	**35.57**
DeepSeek-V3.2	9.88	10.41	11.86	11.33	12.52	**15.81**
Qwen3.5	10.14	10.67	11.59	11.07	12.52	**15.02**

TABLE V: Pass@1 accuracy (%) across four training regimes. **Red**: best per column.

Regime	Qwen-3-8B	GLM-4-9B	LLaMA3-8B
Backbone	2.53	2.24	1.84
+Sim.	1.97	1.65	1.32
+Real	5.84	4.93	3.76
+Sim.→Real	**7.63**	**6.52**	**5.21**

TABLE VI: Impact of operation types on Pass@1 (%). Each row removes one operation type (✗). ↓: degradation, ↑: improvement relative to the full IDE interaction trace.

Setting	Claude 4.6	Gemini 3.1	MiniMax
Full sequence	13.57	11.46	2.77
Navigation & Selection			
✗ cursor_sel.	12.78 ↓0.79	11.20 ↓0.26	3.16 ↑0.39
✗ copy/paste	13.04 ↓0.53	11.20 ↓0.26	2.90 ↑0.13
✗ view switching	12.52 ↓1.05	10.67 ↓0.79	3.03 ↑0.26
Execution & Editing			
✗ terminal	11.86 ↓1.71	9.88 ↓1.58	2.37 ↓0.40
✗ edit	4.22 ↓9.35	3.16 ↓8.30	0.66 ↓2.11
AI Interaction			
✗ agent_req.	13.31 ↓0.26	11.33 ↓0.13	2.64 ↓0.13

Key finding ❸: *Although LLM-generated simulated data cannot substitute for real-world data, the two complement each other—their combination yields performance gains beyond either source alone.*

VI. Discussion

A. Ablation study

To understand how different operation types contribute to proactive intent prediction, we conduct an operation-type ablation study. Starting from the full IDE interaction trace, we remove one operation type at a time and measure the resulting change in Pass@1. To examine whether different models use these signals in different ways, we evaluate three models with different performance levels: Claude Sonnet 4.6, Gemini 3.1 Pro, and MiniMax-M2.5. Results are reported in Table VI.

Comparison across operation types. As shown in Table VI, removing `edit` causes the largest performance drop across all three models, decreasing Pass@1 by 9.35, 8.30, and 2.11 percentage points, respectively. Removing `terminal_execution` also leads to degradation. These results indicate that code edits and execution feedback provide the most direct information for predicting developer intent. In contrast, removing `agent_request` has only a minor effect, likely because this operation type appears less frequently. Overall, current models rely heavily on edit and execution signals, while many other behavioral signals in real-world IDE interaction traces remain underutilized.

Sensitivity across models. Models with different capabilities show different sensitivity to operation-type removal. Claude Sonnet 4.6 degrades under every ablation setting, suggesting that it can use a broader range of IDE signals. In particular, navigation and selection operations provide complementary context for stronger models. By contrast, MiniMax-M2.5 slightly improves when `cursor_selection`, `copy/paste`, or `view switching` is removed, suggesting that less capable models may treat these signals as noise rather than useful information.

Key finding ❹: *Current models rely heavily on explicit execution information, while stronger models can also use contextual signals from other operations. This suggests that effectively leveraging developers' IDE interaction traces remains a key challenge for proactive intent prediction.*

B. Threats to validity

Limitations of intent annotation. Developer intent in real-world IDE interaction traces is implicit and cannot be directly observed. Therefore, our annotation pipeline relies on an LLM and observable behavioral signals, such as substantial code edits and explicit AI-assistant requests, to infer candidate intents. This process may miss some intents that would be valuable for proactive assistance. To mitigate this threat, domain experts independently review the candidate intents in the final stage of the pipeline, correct inaccurate intent, and recover valid candidates that were incorrectly filtered out. Nevertheless, accurately recovering developer intent from IDE interaction traces remains an open challenge.

Data collection scope. Limited by the cost and privacy risks of collecting real-world IDE interaction traces, it is impractical to exhaustively cover all development time scales and developer populations. We mitigate this threat by recruiting 1,246 experienced industry developers from five major development scenarios, including frontend, backend, full-stack, database, and algorithm engineering, and by collecting traces from their

own active projects. This design provides broad coverage of real-world daily development behavior.

Restricted release for privacy. The IDE interaction traces in ProCodeBench may contain sensitive commercial information, such as proprietary enterprise repositories. Although we obtained consent from volunteers through a data collection agreement, the agreement does not permit unrestricted public release of the collected data. Therefore, we cannot fully open-source the raw dataset. To support reproducibility, we instead provide a controlled-access evaluation platform for academic research. Researchers may apply for access with institutional information; once approved, they can submit their model or agent system and obtain evaluation results on ProCodeBench.

VII. Conclusion

This paper investigates whether LLM-generated simulated data can faithfully support proactive code assistance in real-world development scenarios. To analyze this question, we first collect large-scale real-world IDE interaction traces from 1,246 volunteers through a VS Code extension, and pair each real-world trace with an LLM-generated simulated counterpart. This paired design allows us to directly examine the simulation-to-reality gap in developer behavior. Our analysis shows that LLM-generated simulated traces differ substantially from real-world traces in behavioral diversity, temporal structure, and exploratory operations, indicating that simulation alone cannot fully capture how developers work in practice.

Building on the collected real-world data, we construct ProCodeBench, a standardized benchmark for proactive intent prediction in real-world development scenarios. Experiments on LLMs, Retrieval-Augmented LLMs, and LLM-based Agents show that existing models still struggle on this benchmark, suggesting that simulation-based evaluation may overestimate intent-prediction ability. We further study how real-world and LLM-generated simulated data affect training. The results show that LLM-generated simulated data alone does not transfer well to real-world development scenarios, but can provide useful initialization before training on real-world data. These findings suggest that future proactive coding assistants should be evaluated on real-world developer behavior and should learn to use the rich but noisy signals contained in IDE interaction traces.

Acknowledgment

References

[1] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. D. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.

[2] Y. Li, D. Choi, J. Chung, N. Kushman, J. Schrittwieser, R. Leblond, T. Eccles, J. Keeling, F. Gimeno, A. Dal Lago, *et al.*, "Competition-level code generation with alphacode," *Science*, vol. 378, no. 6624, pp. 1092–1097, 2022.

[3] M. Schäfer, S. Nadi, A. Eghbali, and F. Tip, "An empirical evaluation of using large language models for automated unit test generation," *IEEE Transactions on Software Engineering*, vol. 50, no. 1, pp. 85–105, 2023.

[4] A. Sergeyuk, E. Huang, D. Karaeva, A. Serova, Y. Golubev, and I. Ahmed, "Evolving with ai: A longitudinal analysis of developer logs," *arXiv preprint arXiv:2601.10258*, 2026.

[5] B. Puryear and G. Sprint, "Github copilot in the classroom: learning to code with ai assistance," *Journal of Computing Sciences in Colleges*, vol. 38, no. 1, pp. 37–47, 2022.

[6] S. Barke, M. B. James, and N. Polikarpova, "Grounded copilot: How programmers interact with code-generating models," *Proceedings of the ACM on Programming Languages*, vol. 7, no. OOPSLA1, pp. 85–111, 2023.

[7] J. T. Liang, C. Yang, and B. A. Myers, "A large-scale survey on the usability of ai programming assistants: Successes and challenges," in *Proceedings of the 46th IEEE/ACM international conference on software engineering*, pp. 1–13, 2024.

[8] R. Khojah, M. Mohamad, P. Leitner, and F. G. de Oliveira Neto, "Beyond code generation: An observational study of chatgpt usage in software engineering practice," *Proceedings of the ACM on Software Engineering*, vol. 1, no. FSE, pp. 1819–1840, 2024.

[9] M. Bruch, M. Monperrus, and M. Mezini, "Learning from examples to improve code completion systems," in *Proceedings of the 7th joint meeting of the European software engineering conference and the ACM SIGSOFT symposium on the foundations of software engineering*, pp. 213–222, 2009.

[10] V. Raychev, M. Vechev, and E. Yahav, "Code completion with statistical language models," in *Proceedings of the 35th ACM SIGPLAN conference on programming language design and implementation*, pp. 419–428, 2014.

[11] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. R. Narasimhan, and O. Press, "SWE-agent: Agent-computer interfaces enable automated software engineering," in *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024.

[12] V. Chen, A. Zhu, S. Zhao, H. Mozannar, D. Sontag, and A. Talwalkar, "Need help? designing proactive ai assistants for programming," in *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*, pp. 1–18, 2025.

[13] S. Zhao, A. Zhu, H. Mozannar, D. Sontag, A. Talwalkar, and V. Chen, "Codinggenie: A proactive llm-powered programming assistant," in *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering*, pp. 1168–1172, 2025.

[14] N. Tang, C. Chen, Z. Fang, G. Xu, M. Dhakal, Y. Shi, C. McMillan, Y. Huang, and T. J.-J. Li, "Programming by chat: A large-scale behavioral analysis of 11,579 real-world ai-assisted ide sessions," *arXiv preprint arXiv:2604.00436*, 2026.

[15] N. Kuo, A. Sergeyuk, V. Chen, and M. Izadi, "Developer interaction patterns with proactive ai: A five-day field study," *arXiv preprint arXiv:2601.10253*, 2026.

[16] H. Mozannar, G. Bansal, A. Fourney, and E. Horvitz, "Reading between the lines: Modeling user behavior and costs in ai-assisted programming," in *Proceedings of the 2024 CHI conference on human factors in computing systems*, pp. 1–16, 2024.

[17] Y. Lu, S. Yang, C. Qian, G. Chen, Q. Luo, Y. Wu, H. Wang, X. Cong, Z. Zhang, Y. Lin, *et al.*, "Proactive agent: Shifting llm agents from reactive responses to active assistance," *arXiv preprint arXiv:2410.12361*, 2024.

[18] J. Kim, J. Choi, W. Chay, D. Kyung, Y. Kwon, Y. Jo, and E. Choi, "Propersim: Developing proactive and personalized ai assistants through user-assistant simulation," *arXiv preprint arXiv:2509.21730*, 2025.

[19] F. Zhang, B. Chen, Y. Zhang, J. Keung, J. Liu, D. Zan, Y. Mao, J.-G. Lou, and W. Chen, "Repocoder: Repository-level code completion through iterative retrieval and generation," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 2471–2484, 2023.

[20] S. Ouyang, W. Yu, K. Ma, Z. Xiao, Z. Zhang, M. Jia, J. Han, H. Zhang, and D. Yu, "Repograph: Enhancing ai software engineering with repository-level code graph," in *13th International Conference on Learning Representations, ICLR 2025*, pp. 30361–30384, International Conference on Learning Representations, ICLR, 2025.

[21] M. Du, B. Xu, C. Zhu, S. Wang, P. Wang, X. Wang, and Z. Mao, "A-rag: Scaling agentic retrieval-augmented generation via hierarchical retrieval interfaces," *arXiv preprint arXiv:2602.03442*, 2026.

[22] J. Wang and Y. Chen, "A review on code generation with llms: Application and evaluation," in *2023 IEEE International Conference on Medical Artificial Intelligence (MedAI)*, pp. 284–289, IEEE, 2023.

[23] C. S. Xia, Y. Wei, and L. Zhang, "Automated program repair in the era of large pre-trained language models," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, pp. 1482–1494, IEEE, 2023.

[24] Y. Ding, Z. Wang, W. U. Ahmad, H. Ding, M. Tan, N. Jain, M. K. Ramanathan, R. Nallapati, P. Bhatia, D. Roth, and B. Xiang, "CrossCodeEval: A diverse and multilingual benchmark for cross-file code completion," in *Thirty-seventh Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2023.

[25] Y. Deng, W. Lei, W. Lam, and T.-S. Chua, "A survey on proactive dialogue systems: Problems, methods, and prospects," *arXiv preprint arXiv:2305.02750*, 2023.

[26] X. Zhou, W. Sun, Q. Ma, Y. Xie, J. Liu, W. Du, S. Welleck, Y. Yang, G. Neubig, S. T. Wu, *et al.*, "Mind the sim2real gap in user simulation for agentic tasks," *arXiv preprint arXiv:2603.11245*, 2026.

[27] Y. Tang, H. Tang, T. Cao, L. Nguyen, A. Zhang, X. Cao, C. Liu, W. Ding, and Y. Li, "ProAgentBench: Evaluating llm agents for proactive assistance with real-world data," *arXiv preprint arXiv:2602.04482*, 2025.

[28] Y. Chai, S. Tang, H. Xiao, R. Liu, and H. Li, "Pirabench: A transition from reactive gui agents to gui-based proactive intent recommendation agents," *arXiv preprint arXiv:2603.08013*, 2026.

[29] J. Li, G. Li, Y. Zhao, Y. Li, H. Liu, H. Zhu, L. Wang, K. Liu, Z. Fang, L. Wang, *et al.*, "Deveval: A manually-annotated code generation benchmark aligned with real-world code repositories," in *Findings of the Association for Computational Linguistics: ACL 2024*, pp. 3603–3614, 2024.

[30] J. Li, G. Li, X. Zhang, Y. Zhao, Y. Dong, Z. Jin, B. Li, F. Huang, and Y. Li, "Evocodebench: An evolving code generation benchmark with domain-specific evaluations," *Advances in Neural Information Processing Systems*, vol. 37, pp. 57619–57641, 2024.

[31] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. J. Cai, M. Terry, Q. V. Le, and C. Sutton, "Program synthesis with large language models," *arXiv preprint arXiv:2108.07732*, 2021.

[32] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. R. Narasimhan, "SWE-bench: Can language models resolve real-world github issues?," in *The Twelfth International Conference on Learning Representations*, 2024.

[33] H. Husain, H.-H. Wu, T. Gazit, M. Allamanis, and M. Brockschmidt, "CodeSearchNet challenge: Evaluating the state of semantic code search," *arXiv preprint arXiv:1909.09436*, 2019.

[34] B. Li, W. Wu, Z. Tang, L. Shi, J. Yang, J. Li, S. Yao, C. Qian, B. Hui, Q. Zhang, *et al.*, "DevBench: A comprehensive benchmark for software development," *arXiv preprint arXiv:2403.08604*, 2024.

[35] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. Xing, *et al.*, "Judging llm-as-a-judge with mt-bench and chatbot arena," *Advances in neural information processing systems*, vol. 36, pp. 46595–46623, 2023.

[36] P. Wang, L. Li, L. Chen, Z. Cai, D. Zhu, B. Lin, Y. Cao, L. Kong, Q. Liu, T. Liu, *et al.*, "Large language models are not fair evaluators," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 9440–9450, 2024.

[37] GLM-5-Team, A. Zeng, X. Lv, Z. Hou, Z. Du, Q. Zheng, B. Chen, D. Yin, C. Ge, C. Huang, C. Xie, C. Zhu, C. Yin, C. Wang, G. Pan, H. Zeng, H. Zhang, H. Wang, H. Chen, J. Zhang, J. Jiao, J. Guo, J. Wang, J. Du, J. Wu, K. Wang, L. Li, L. Fan, L. Zhong, M. Liu, M. Zhao, P. Du, Q. Dong, R. Lu, Shuang-Li, S. Cao, S. Liu, T. Jiang, X. Chen, X. Zhang, X. Huang, X. Dong, Y. Xu, Y. Wei, Y. An, Y. Niu, Y. Zhu, Y. Wen, Y. Cen, Y. Bai, Z. Qiao, Z. Wang, Z. Wang, Z. Zhu, Z. Liu, Z. Li, B. Wang, B. Wen, C. Huang, C. Cai, C. Yu, C. Li, C. Hu, C. Zhang, D. Zhang, D. Lin, D. Yang, D. Wang, D. Ai, E. Zhu, F. Yi, F. Chen, G. Wen, H. Sun, H. Zhao, H. Hu, H. Zhang, H. Liu, H. Zhang, H. Peng, H. Tai, H. Zhang, H. Liu, H. Wang, H. Yan, H. Ge, H. Liu, H. Chu, J. Zhao, J. Wang, J. Zhao, J. Ren, J. Wang, J. Zhang, J. Gui, J. Zhao, J. Li,

J. An, J. Li, J. Yuan, J. Du, J. Liu, J. Zhi, J. Duan, K. Zhou, K. Wei, K. Wang, K. Luo, L. Zhang, L. Sha, L. Xu, L. Wu, L. Ding, L. Chen, M. Li, N. Lin, P. Ta, Q. Zou, R. Song, R. Yang, S. Tu, S. Yang, S. Wu, S. Zhang, S. Li, S. Li, S. Fan, W. Qin, W. Tian, W. Zhang, W. Yu, W. Liang, X. Kuang, X. Cheng, X. Li, X. Yan, X. Hu, X. Ling, X. Fan, X. Xia, X. Zhang, X. Zhang, X. Pan, X. Zou, X. Zhang, Y. Liu, Y. Wu, Y. Li, Y. Wang, Y. Zhu, Y. Tan, Y. Zhou, Y. Pan, Y. Zhang, Y. Su, Y. Geng, Y. Yan, Y. Tan, Y. Bi, Y. Shen, Y. Yang, Y. Li, Y. Liu, Y. Wang, Y. Li, Y. Wu, Y. Zhang, Y. Duan, Y. Zhang, Z. Liu, Z. Jiang, Z. Yan, Z. Zhang, Z. Wei, Z. Chen, Z. Feng, Z. Yao, Z. Chai, Z. Wang, Z. Zhang, B. Xu, M. Huang, H. Wang, J. Li, Y. Dong, and J. Tang, "Glm-5: from vibe coding to agentic engineering," 2026.

[38] Qwen Team, "Qwen3.5: Towards native multimodal agents," February 2026.

[39] S. Zhang, Y. Ding, S. Lian, S. Song, and H. Li, "Coderag: Finding relevant and necessary knowledge for retrieval-augmented repository-level code completion," in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 23289–23299, 2025.

[40] W. Liu, A. Yu, D. Zan, B. Shen, W. Zhang, H. Zhao, Z. Jin, and Q. Wang, "Graphcoder: Enhancing repository-level code completion via coarse-to-fine retrieval based on code context graph," in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, pp. 570–581, 2024.

[41] Q. Team, "Qwen3 technical report," 2025.

[42] Team GLM, A. Zeng, B. Xu, B. Wang, C. Zhang, D. Yin, D. Rojas, G. Feng, H. Zhao, H. Lai, H. Yu, H. Wang, J. Sun, J. Zhang, J. Cheng, J. Gui, J. Tang, J. Zhang, J. Li, L. Zhao, L. Wu, L. Zhong, M. Liu, M. Huang, P. Zhang, Q. Zheng, R. Lu, S. Duan, S. Zhang, S. Cao, S. Yang, W. L. Tam, W. Zhao, X. Liu, X. Xia, X. Zhang, X. Gu, X. Lv, X. Liu, X. Liu, X. Yang, X. Song, X. Zhang, Y. An, Y. Xu, Y. Niu, Y. Yang, Y. Li, Y. Bai, Y. Dong, Z. Qi, Z. Wang, Z. Yang, Z. Du, Z. Hou, and Z. Wang, "Chatglm: A family of large language models from glm-130b to glm-4 all tools," 2024.

[43] A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Vaughan, *et al.*, "The llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024.